

TruthLens (News Validator) - Complete Interview Preparation Guide

Project: TruthLens / News Validator Web App

Stack: React + Vite + Django REST + PostgreSQL + Redis + Celery

Document version: 1.0 - May 2026

Table of Contents

1. [Project Overview](#1-project-overview)
 2. [Complete App Flow](#2-complete-app-flow)
 3. [Tech Stack](#3-tech-stack)
 4. [PostgreSQL Deep Dive](#4-postgresql-deep-dive)
 5. [Redis Deep Dive](#5-redis-deep-dive)
 6. [News Verification Logic](#6-news-verification-logic)
 7. [Authentication & Security](#7-authentication--security)
 8. [Backend Architecture & API Flow](#8-backend-architecture--api-flow)
 9. [Important Folders & Files](#9-important-folders--files)
 10. [Scalability & Production](#10-scalability--production)
 11. [Challenges & Tradeoffs](#11-challenges--tradeoffs)
 12. [If PostgreSQL or Redis Removed](#12-if-postgresql-or-redis-removed)
 13. [20 Interview Q&A](#13-20-interview-qa)
 14. [2-Minute Pitch](#14-2-minute-pitch)
 15. [Quick Reference Card](#15-quick-reference-card)
-

1. Project Overview

Real-world problem

People share news screenshots, videos, and text on social media. Much of it is fake or misleading. TruthLens lets a user:

1. Upload an **image**, **video**, or **text claim**
2. The system **reads** the content (OCR / speech-to-text)
3. It **searches real news** about that claim
4. It returns a **confidence score**, **verdict** (Likely True / Uncertain / Likely False / Unverifiable), **summary**, and **sources**

Real-world value: Helps users decide whether to trust something before sharing it - like a lightweight fact-check assistant, not a court of law.

What TruthLens is NOT

* Not a legal fact-check authority

- * Not a large language model that "knows" truth
 - * Not storing full news articles long-term - it fetches evidence on demand
-

2. Complete App Flow

Frontend -> Backend (step by step)

```
User (Browser)
|
v
React UI (localhost:3000) - Material UI
|  upload file/text OR paste text
v
POST /api/upload/          -> saves Upload in PostgreSQL (status: pending)
|
v
POST /api/verify/         -> starts background pipeline (status: processing)
|
v
GET /api/upload/{id}/      -> poll every 2.5s until status = completed
|
v
Show result: score %, verdict, summary, sources, flags
```

Backend pipeline (`tasks/verification\_tasks.py`)

```
| Step | What happens |
| 1. Extract | Image -> Tesseract OCR; Video -> ffmpeg + Whisper; Text -> direct |
| 2. Search news | GNews API + Google News RSS (2 sources, parallel) |
| 3. Score | Keywords + similarity + trusted-source weights |
| 4. Save | VerificationResult in PostgreSQL, Upload.status = completed |
```

Note: In development, verification often runs in a Python background thread. Celery + Redis are wired for production async processing.

3. Tech Stack

```
| Layer | Technology | Why chosen |
| Frontend | React 18 + Vite | Fast dev, component UI, SPA |
| UI | Material UI | Professional components out of the box |
| HTTP | Axios | Simple API calls + error handling |
| Backend | Django 4.2 | ORM, admin, auth, mature ecosystem |
| API | Django REST Framework | REST endpoints, serializers, validation |
| Auth | SimpleJWT | Stateless JWT access + refresh tokens |
| Database | PostgreSQL | Reliable relational DB for uploads & results |
| Task queue | Celery + Redis | Async verification jobs at scale |
| OCR | Tesseract (pytesseract) | Free, local text from images |
| Speech | OpenAI Whisper | Local transcription from video |
| News | GNews API + Google News RSS | External evidence; keys stay server-side |
```

| Config | python-decouple | Secrets in .env, not in source code |

AI / NLP usage (honest answer)

Component	Technology	Purpose
OCR	Tesseract	Read text from images
STT	Whisper + ffmpeg	Transcribe video audio
Scoring	Keywords + Jaccard/overlap similarity	Match claim to articles
Summary	Template engine (`summarizer.py`)	Human-readable result text
Fact-check LLM	****Not used****	By design - explainable + low cost

4. PostgreSQL Deep Dive

Where PostgreSQL is used

- * ****Upload**** - every submission (file metadata, extracted text, status)
- * ****VerificationResult**** - one result per upload (score, verdict, summary, sources JSON, flags JSON)
- * ****Django User**** - optional link from upload to user
- * ****django_celery_results**** - Celery task results (when worker used)
- * ****JWT blacklist tables**** - revoked refresh tokens

Tables & relations

User (Django auth)
|
| 1 -- * (optional, SET NULL on delete)
v
Upload (UUID primary key)
|
| 1 -- 1 (CASCADE on delete)
v
VerificationResult (UUID primary key)

Key columns

Upload: file_type, raw_text, extracted_text, status (pending -> processing -> completed/failed), mime_type, file_size

VerificationResult: confidence_score (0.0-1.0), verdict, summary, sources (JSON), flags (JSON)

Example ORM queries

```
# Create upload
Upload.objects.create(file_type="image", ...)

# Poll with JOIN (one query)
Upload.objects.select_related("result").get(pk=upload_id)

# Upsert result
VerificationResult.objects.update_or_create(upload=upload, defaults={...})
```

Indexes

```
models.Index(fields=["status"])
models.Index(fields=["file_type"])
models.Index(fields=["user", "-uploaded_at"])
```

Why: Fast polling by status; fast user history for future dashboards.

ACID properties (interview answer)

Property	Example in TruthLens
Atomicity	Upload + Result save succeed or fail together in a transaction
Consistency	Foreign keys enforce Upload ? Result integrity
Isolation	Concurrent polls don't corrupt partial writes
Durability	Completed verifications survive server restart

Why PostgreSQL over MongoDB?

PostgreSQL	MongoDB
Strong schema for upload lifecycle	Flexible but weaker for strict status machine
Relational User ? Upload ? Result	Embedding works; relational queries are awkward
JSONField for sources/flags when needed	Native JSON, but overkill for core entities
ACID + Django ORM first-class	Better for logs/docs than transactional pipeline

Interview line: "PostgreSQL for transactional structured data; JSONField only where the payload is semi-structured."

5. Redis Deep Dive

Where Redis is configured

```
REDIS_URL=redis://localhost:6379/0
CELERY_BROKER_URL = REDIS_URL
```

Uses in TruthLens

Use	How
Celery broker	Queue `run_verification_pipeline` tasks to workers
Rate limiting	GNews daily call counter (`gnews:daily_calls`, TTL 86400s) via Django cache
Sessions	Not primary - JWT is stateless
Performance	Keeps long OCR/Whisper/news search off the HTTP thread

Production note: Point CACHES to Redis so rate limits work across multiple app servers.

Key concepts

- * **Caching:** Store GNews API call count to respect 100 requests/day (free tier)
- * **TTL:** 24-hour expiry on rate-limit keys
- * **Broker:** Redis lists/streams hold Celery task messages

* **Not in browser:** Redis is backend-only; users never touch it

6. News Verification Logic

Pipeline overview

1. **Extract** claim text (OCR / STT / passthrough)
2. **Build search query** (`core/search\_query.py`) - clean noisy OCR
3. **Fetch articles** - GNews + Google News RSS in parallel
4. **Score** - similarity + keywords + trusted outlets
5. **Summarize** - template-based natural language
6. **Persist** - PostgreSQL

Sources (current - only 2)

Source	API key?	Role
GNews	Yes (`GNEWS_API_KEY`)	Primary news API
Google News RSS	No	Secondary trusted headlines

Removed: Wikipedia (loose matches inflated false confidence)

Confidence scoring (`apps/scoring/engine.py`)

For each article:

1. **Similarity** - Jaccard + overlap coefficient
2. **Keyword overlap** - minimum 2 shared keywords (or high similarity)
3. **Trusted source bonus** - Reuters, BBC, etc. from credibility map
4. **Normalize** raw score ÷ ceiling
5. **Cap** at **82%** maximum
6. **Verdict** - LIKELY_TRUE only if score ≥ 65% AND ≥ 2 trusted sources

Verdicts

Verdict	Meaning
LIKELY_TRUE	Strong corroboration from trusted news
UNCERTAIN	Some evidence, not conclusive
LIKELY_FALSE	Weak or contradicting evidence
UNVERIFIABLE	No text or no matching sources

Fake news detection (what to say)

"We measure whether trusted news coverage corroborates the claim. We don't claim legal truth - we show evidence and a conservative score."

7. Authentication & Security

Configured

- * JWT: `/api/auth/login/`, `/api/auth/refresh/`, `/api/auth/logout/`
- * Access token: 60 minutes; Refresh: 7 days with rotation + blacklist
- * API keys in `.env` only - never in React bundle

Current development state

Main verify/upload endpoints use AllowAny (no login required in UI).

Security measures implemented

- * Sanitized API errors (`core/exceptions.py`, `core/security.py`)
- * No stack traces or API key names in production responses
- * `.gitignore` excludes `.env`
- * Production: `DEBUG=False`, HTTPS, HSTS, secure cookies

Production recommendations

- * Require JWT on `/api/upload/` and `/api/verify/`
- * Rate limit per IP/user
- * Rotate secrets via vault (not flat `.env` on server)

8. Backend Architecture & API Flow

Architecture diagram

```
[React SPA] --REST--> [Django + DRF]
|
+-----?-----?
v             v             v
[PostgreSQL]  [Media files]  [External APIs]
Upload/Result  local/S3      GNews, Google RSS
?
| (optional)
[Celery Worker] ?-- [Redis broker]
```

Main API endpoints

Method	Endpoint	Purpose
POST	/api/upload/	Create upload
POST	/api/verify/	Start verification pipeline
GET	/api/upload/{id}/	Poll status + result
GET	/api/result/{id}/	Get result by ID
POST	/api/auth/login/	JWT login
POST	/api/fact-check/search/	Standalone news search

| POST | `/api/extract/image/` | OCR only (testing) |

Django apps (modular monolith)

App	Responsibility
`verifier`	Upload, Result models, main API
`extraction`	OCR, STT, dispatcher
`fact\_check`	News sources, aggregator
`scoring`	Engine, keywords, flags, summarizer

9. Important Folders & Files

```
TruthLens/
+-- frontend/
|   src/api/client.js, verifier.js      -> HTTP to backend
|   src/hooks/useVerification.js        -> upload -> verify -> poll
|   src/pages/HomePage.jsx             -> main UI
|   src/components/results/            -> score, verdict, sources
|   vite.config.js                     -> proxy /api -> :8000
|
+-- backend/
|   config/settings/base.py             -> DB, JWT, Celery, keys
|   config/urls.py                     -> routes
|   apps/verifier/                     -> Upload, Result, views
|   apps/extraction/                   -> OCR, STT, dispatcher
|   apps/fact_check/                   -> GNews, RSS, aggregator
|   apps/scoring/                      -> engine, summarizer
|   core/                              -> security, search_query, http
|   tasks/verification_tasks.py         -> full pipeline
|   manage.py                          ->
|   .env                               -> secrets (NEVER commit)
```

10. Scalability & Production

Area	Development	Production
Async	Background thread	Celery workers + Redis
Media	Local `media/`	S3 + CDN
Database	Single Postgres	Pooling, replicas
Secrets	`.env` file	AWS Secrets Manager / Vault
Auth	AllowAny	JWT required
Cache	May be in-memory	Redis for all caches
Whisper/OCR	Same server	Dedicated worker / GPU
Monitoring	Console logs	Sentry, Prometheus

11. Challenges & Tradeoffs

| Challenge | Solution |

OCR noise on screenshots	Image preprocessing + multi-PSM Tesseract
100% confidence on fake news	Removed Wikipedia; stricter matching; 82% cap
GNews free tier limits	Short queries, daily rate limit cache
Long processing time	Async pipeline + frontend polling
Windows SSL errors	certifi + HTTP_SSL_VERIFY flag

Tradeoff: Rule-based scoring is explainable and cheap but less nuanced than LLM+RAG.

12. If PostgreSQL or Redis Removed

Removed	Impact
PostgreSQL	No persistent uploads/results; polling breaks; need SQLite (limited) or lose all data on restart
Redis	No Celery queue; must use threads (doesn't scale); distributed rate limiting fails across servers

13. 20 Interview Q&A

Q1. Tell me about your project.

TruthLens is a full-stack misinformation verification tool. Users submit images, videos, or text; we extract the claim, search GNews and Google News RSS, score overlap against trusted outlets, and return confidence, verdict, and sources.

Q2. Why Django?

ORM, migrations, admin, auth, and Python's ML/OCR ecosystem (Tesseract, Whisper) in one stack.

Q3. Why PostgreSQL?

Relational upload lifecycle, ACID, joins for polling, JSONField for flexible source/flag payloads.

Q4. How is Redis used?

Celery message broker; Django cache for GNews daily quota (TTL 24h). Production should use Redis for cache too.

Q5. Walk through one request.

POST upload -> POST verify -> poll GET upload until status=completed with embedded result.

Q6. How do you detect fake news?

Evidence-based corroboration: similarity + keyword overlap + trusted publisher weights. No single "fake classifier" ML model.

Q7. Do you use AI?

Whisper + Tesseract for extraction. Scoring uses keyword/similarity math. Summaries are templates, not GPT.

Q8. Explain the confidence score.

Raw points normalized and capped at 82%. LIKELY_TRUE needs high score plus 2+ trusted sources.

Q9. Why only two news sources?

Reduce noise and API cost. Wikipedia caused false high scores.

Q10. How do you secure API keys?

Server-side .env only. Frontend calls /api proxy. Sanitized error messages.

Q11. Authentication?

SimpleJWT configured. Verify endpoints currently AllowAny for demo; production would require JWT.

Q12. Celery's role?

Async run_verification_pipeline so web requests don't block during OCR/Whisper/news fetch.

Q13. Database indexes?

status, file_type, (user, uploaded_at) - for polling and user history.

Q14. ACID example?

Result + upload status update persist atomically - no "completed" without a result row.

Q15. PostgreSQL vs MongoDB?

Postgres for relations + status machine; MongoDB poor fit for 1:1 Upload->Result integrity.

Q16. OCR fails?

Empty text -> UNVERIFIABLE, NO_TEXT flag, user-friendly message.

Q17. Scale to 10k users/day?

Celery workers, Redis, S3 media, DB pooling, rate limits, separate ML workers.

Q18. Biggest bug fixed?

Fake claims scoring 100% - removed Wikipedia, tightened scoring, capped confidence.

Q19. Next improvements?

Require auth, Redis cache everywhere, Google Fact Check API, automated scoring tests.

Q20. Test verification quality?

Manual fake vs real claim cases; inspect matched articles in logs; tune engine constants.

14. 2-Minute Pitch

*"I built **TruthLens**, a full-stack news verification web application. The problem is misinformation spread through screenshots and viral posts - users need a quick way to check if a claim is supported by real news.*

*The **frontend** is React with Material UI and Vite. Users upload an image, video, or text. The **backend** is Django REST Framework with **PostgreSQL** storing uploads and verification results in a clear lifecycle: pending, processing, completed.*

*When they hit verify, we run a **pipeline**: extract text using **Tesseract** for images or **Whisper** for video, build a focused search query, then fetch articles from **GNews** and **Google News RSS** in parallel. We score the claim using **keyword overlap**, **text similarity**, and a **credibility map** of trusted outlets - not a black-box LLM, so results are explainable.*

*The API returns a **confidence score** capped at 82%, a **verdict**, a natural-language **summary**, and **source links**. We use **JWT** for auth infrastructure, keep all API keys server-side, and designed for **Celery and Redis** to run verification asynchronously in production.*

The hardest part was tuning scoring so fake claims don't get false high confidence - we removed noisy sources, tightened matching rules, and require multiple trusted corroborations before marking something Likely True."

15. Quick Reference Card

Topic	One-line answer
Problem	Verify news claims from image/video/text
DB	PostgreSQL - Upload + VerificationResult (1:1)
Redis	Celery broker + rate-limit cache
Sources	GNews + Google News RSS only
AI	Whisper + Tesseract; scoring = rules
Max score	82% by design
Auth	JWT wired; UI public in dev
Async	Thread (dev); Celery+Redis (prod)

End of document - TruthLens Interview Guide